

Unidade de Aprendizagem: Sistemas Interativos e de Visualização

Professor: Edson Moacir Ahlert

Estudantes: Kauan Morinel Calheiro e Everton Luiz de Oliveira

Data: 10/08/2026 | **Plataforma:** n8n 2.27.4 (self-hosted, Docker)

1. Qual problema real o sistema pretende resolver?

Convite de evento raramente chega estruturado. Chega como cartaz: um JPEG anexado a um e-mail, um PDF de programação. A informação está toda lá (data, horário, local), só que em pixels, e alguém precisa abrir, ler e digitar na agenda. Esse passo manual é o que faz o convite ser adiado e depois esquecido.

Criamos um fluxo no n8n que monitora a caixa de entrada, lê o anexo com um modelo multimodal, estrutura os campos do evento, grava no Google Calendar e confirma no Telegram. O usuário encaminha o e-mail e não faz mais nada.

2. Qual modalidade foi escolhida e por que ela é adequada?

Modalidade central: **processamento de imagem com leitura de texto (OCR visual)**. O cartaz não tem texto selecionável, tem título em fonte decorativa e horários espalhados em blocos visuais, ao lado de ícones e de um QR Code. Extrair "17 de agosto, 14h às 21h30, Centro de Convivência Prédio 9" desse layout exige interpretar a imagem, não procurar strings nela. Nenhuma abordagem de texto puro cobre o caso.

3. Onde o OCR ocorre e qual informação é extraída?

No módulo 4, nó `Analyze an image`, com `gemini-3-flash-preview` em modo `image/analyze` lendo o binário do anexo. Ele não devolve campos prontos, e sim a transcrição bruta do cartaz: título, data, cada atividade com seu local e seus horários, e a URL de inscrição embutida no QR Code. A conversão dessa transcrição em campos acontece só no módulo seguinte.

4. A solução utiliza áudio?

Não. Uma versão anterior do fluxo era disparada por Telegram e tinha um ramo de áudio, com transcrição de mensagens de voz pelo Gemini. Ao migrar o gatilho para e-mail, esse ramo foi removido: anexo de áudio em convite é raro, e manter um caminho que não seria testado só adicionaria superfície sem uso. O roteador manteve as três saídas relevantes ao novo gatilho, que são imagem, documento e texto.

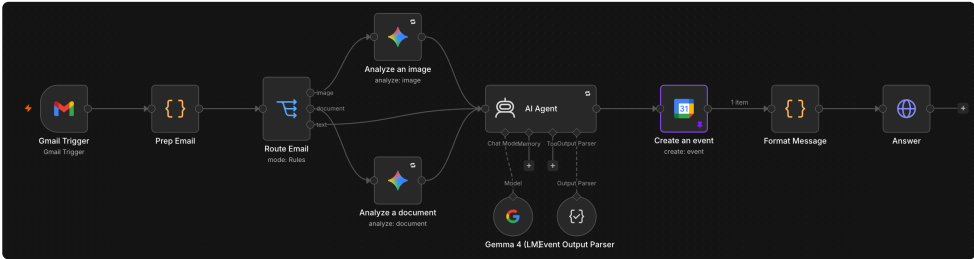
5. Qual é o papel da IA no fluxo?

A IA aparece em dois módulos, com papéis deliberadamente separados:

- **gemini-3-flash-preview** **percebe**. Recebe o binário e devolve a transcrição bruta.
- **gemma-4-31b-it** **estrutura**. Converte a transcrição em JSON validado por schema obrigatório de cinco campos.

A separação evita que um único passo decida ao mesmo tempo o que é relevante e como formatar. Se a percepção já resumisse, um horário descartado ali estaria perdido antes de qualquer validação. Todo o resto do fluxo, roteamento, normalização e formatação, é código determinístico.

6. Quais são os módulos e como eles se relacionam?



Oito módulos conectados, acima do mínimo de cinco exigido:

#	Módulo	Função
1	Gmail Trigger	detecta o e-mail e baixa o anexo
2	Prep Email	normaliza o binário e classifica o tipo
3	Route Email	roteia entre imagem, documento e texto
4	Analyze an image / a document	leitura multimodal (IA)
5	AI Agent + Output Parser	estruturação em JSON (IA)
6	Create an event	grava no Google Calendar
7	Format Message	monta a notificação
8	Answer	envia no Telegram

Os módulos 1 a 3 preparam, o 4 e o 5 interpretam, e o 6 a 8 entregam. As três saídas do roteador convergem de volta no módulo 5, de modo que existe um único ponto de estruturação, independentemente do tipo de anexo.

7. Que informação é transformada entre os módulos?

O anexo chega do Gmail como `attachment_0`. O módulo 2 o renomeia para `data`, que é o campo que todos os nós de IA leem, e classifica o tipo a partir do MIME. Essa renomeação única foi o que permitiu reaproveitar o mesmo nó de imagem quando a origem do fluxo mudou.

Depois do módulo 4 o binário é descartado e só a transcrição segue. O módulo 5 a converte em JSON com `title`, `description`, `startDateTime`, `endDateTime` e `location`. A partir do módulo 6 o que circula não é mais a intenção extraída, e sim o evento como o Google Calendar o devolveu, decisão que se mostrou importante na seção 9. Nada é persistido fora da agenda: o fluxo não mantém banco próprio.

8. Como o usuário interage com o sistema e com os resultados?

A entrada é o gesto que ele já faria de qualquer jeito, que é encaminhar o e-mail. Não há formulário nem interface a aprender.

A saída aparece em duas superfícies com papéis distintos. O **Google Calendar** é a visualização persistente, onde o compromisso passa a existir junto aos demais e dispara lembrete. O **Telegram** é a visualização imediata, que fecha o ciclo confirmando o que foi gravado, sem exigir que o usuário abra a agenda para conferir.

9. Demonstração do funcionamento e resultado final

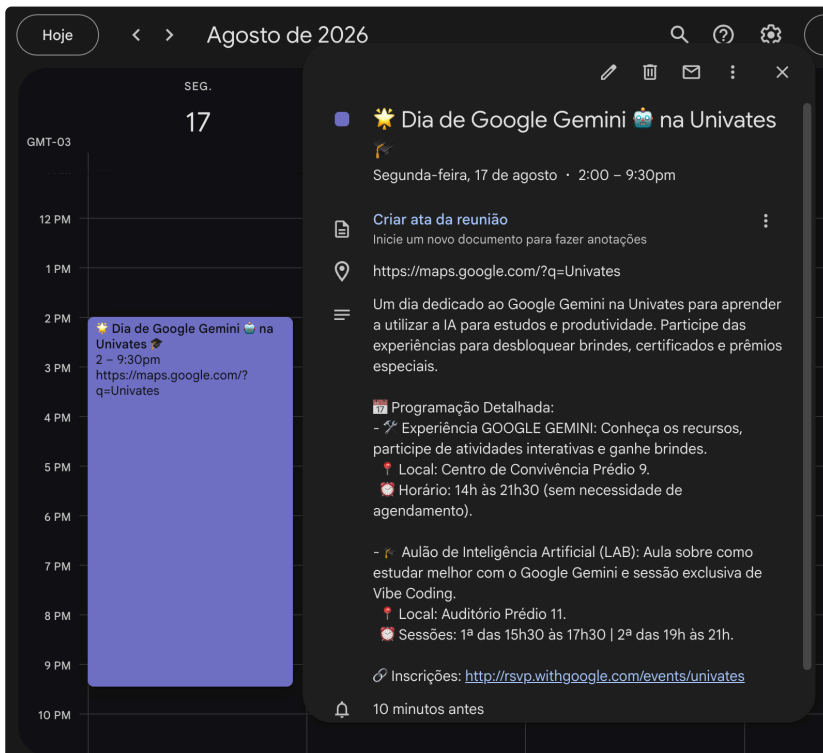
Teste real em 10/08/2026, execução #48, 58 segundos de ponta a ponta.

Enviamos por e-mail o cartaz abaixo, sem nenhum texto no corpo da mensagem:



O gatilho disparou em 1,8 s. O Gemini transcreveu as duas atividades com seus locais e horários, inclusive a URL de inscrição do QR Code. O agente estruturou o JSON em 44 s e o evento foi criado na agenda, com título, local, descrição e data corretos.

O horário, porém, saiu **uma hora adiantado**: o cartaz diz 14h às 21h30 e a automação gravou 15h às 22h30. A causa é que o contêiner subia sem `GENERIC_TIMEZONE`, e o `n8n` assume `America/New_York`, de modo que um `datetime ISO` sem offset foi resolvido no fuso errado. A correção foi feita em duas camadas: fuso fixado em `America/Sao_Paulo` no `docker-compose.yml` e ancoragem determinística em `-03:00` por expressão no nó `Create an event`, para o caso de o modelo devolver data sem offset. O print abaixo mostra o evento já com o horário do cartaz.



Esse defeito é o achado mais instrutivo do trabalho. Ele não gera erro, não aparece em log e não interrompe a execução. Produz um evento plausível, no dia certo, na hora errada. O que o tornou visível foi a notificação do Telegram ser montada a partir da resposta do Google Calendar, e não da saída do agente: confirmar ao usuário aquilo que ele pediu é sempre consistente e nunca informativo.

10. Prompts utilizados

Módulo 4, leitura da imagem (`gemini-3-flash-preview`):

```
Extraia todas as informações possíveis desta imagem e retorne a saída bruta, sem resumos ou filtros.
```

A instrução "sem resumos ou filtros" é o ponto central: o módulo seguinte já não tem acesso à imagem, então qualquer corte feito aqui é irreversível. O nó de documento usa o mesmo prompt trocando "imagem" por "documento".

Módulo 5, estruturação (`gemma-4-31b-it`), resumo do system message:

```
Você é um assistente especializado em extrair informações de eventos.
Extraia, para criar um evento no Google Calendar:
- title: título sucinto, com ao menos 2 emojis contextuais
- description: descrição completa, com todos os detalhes mencionados
- startDateTime / endDateTime: ISO 8601. Use a data de referência fornecida para resolver datas relativas como "amanhã" ou "sexta"
- location: link do Google Maps, ou string vazia se não houver
```

Os cinco campos são obrigatórios no schema do output parser. `location` é string vazia em vez de campo opcional porque campo ausente é indistinguível de campo esquecido.